

**File: \_\_init\_\_.py**

**Directory: C:/Users/sense/PycharmProjects/cnc\_aoil/.venv/Lib/site-packages/gcode\_machine**

"""

gcode-machine - CNC controller state machine for parsing and pre-processing of G-code  
Copyright (C) 2016, 2023 Michael Franzl

This file is part of gcode-machine.

gcode-machine is free software: you can redistribute it and/or modify  
it under the terms of the GNU General Public License as published by  
the Free Software Foundation, either version 3 of the License, or  
(at your option) any later version.

gcode-machine is distributed in the hope that it will be useful,  
but WITHOUT ANY WARRANTY; without even the implied warranty of  
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the  
GNU General Public License for more details.

You should have received a copy of the GNU General Public License  
along with pygltpainter. If not, see <<https://www.gnu.org/licenses/>>.

"""

```
import re
import logging
import math
```

```
class GcodeMachine:
```

```
    """ This implements a simple CNC state machine that can be used
    for simulation and processing of G-code.
```

```

    For usage, see README.md.
```

```

    Callbacks:
```

```

    on_feed_change
    : Emitted when a F keyword is parsed from the G-Code.
    : One argument: the feed rate in mm/min
```

```

    on_var_undefined
    : Emitted when a variable is to be substituted
    : but no substitution value has been set previously.
    : One argument: the key of the undefined variable
```

```
    """
```

```

    def __init__(
        self,
        impos=(0, 0, 0),
        ics="G54",
        cs_offsets={"G54": (0, 0, 0)}
    ):
```

```
);  
""" Initialization.
```

## Parameters

-----

`impos` : 3-tuple  
Initial machine position

`ics` : str  
Initial coordinate system. Default: 'G54'

`cs_offsets` : dict  
Default: {"G54": (0, 0, 0)}  
"""

```
self.logger = logging.getLogger('gcode_machine')
```

```
# @var line  
# Holds the current Gcode line  
self.line = ""
```

```
# @var vars  
# A dict holding values for variable substitution.  
self.vars = {}
```

```
# @var callback method for parent applications  
self.callback = self.__default_callback
```

```
# @var do_feed_override  
# If set to True, F commands will be replaced with the feed  
# speed set in `request_feed`. If set to False, no feed speed  
# processing will be done.  
self.do_feed_override = False
```

```
# @var do_fractionize_lines  
# If set to True, linear movements over the threshold of  
# `fract_linear_threshold` will be broken down into small line  
# segments of length `fract_linear_segment_len`. If set to False  
# no processing is done on lines.  
self.do_fractionize_lines = False
```

```
# @var do_fractionize_arcs  
# If set to True, arcs will be broken up into small line segments.  
# If set to False, no processing is done on arcs.  
self.do_fractionize_arcs = False
```

```
# @var fract_linear_threshold  
# The threshold for the fractionization of lines.  
self.fract_linear_threshold = 1
```

```

# @var fract_linear_segment_len
# The length of the segments of fractionized lines.
self.fract_linear_segment_len = 0.5

# @var spindle_factor
# Scale S by this value
self.spindle_factor = 1

# @var request_feed
# If `do_feed_override` is True, a F command will be inserted
# to the currently active command with this value.
self.request_feed = None

# @var current_feed
# The current feed rate of the machine
self.current_feed = None

# @var contains_feed
# True if the current line contains a F command
self.contains_feed = False

# @var current_distance_mode
# Contains the current distance mode of the machine as string
self.current_distance_mode = "G90"

# @var current_motion_mode
# Contains the current motion mode of the machine as integer (0 to 3)
self.current_motion_mode = 0

# @var current_plane_mode
# Contains the current plane mode of the machine as string
self.current_plane_mode = "G17"

# @var cs_offsets
# Coordinate system offsets. A Python dict with
# 3-tuples as offsets.
self.cs_offsets = cs_offsets

# @var cs
# Current coordinate system as string.
self.cs = ics

# @var pos_m
# Contains the current machine position before execution
# of the currently set line (the target of the last command)
self.pos_m = list(impos) # initial machine position

# @var pos_w
# Contains the current working position before execution
# of the currently set line (the target of the last command)
self.pos_w = [

```

```

        self.pos_m[0] - self.cs_offsets[self.cs][0],
        self.pos_m[1] - self.cs_offsets[self.cs][1],
        self.pos_m[2] - self.cs_offsets[self.cs][2]
    ]

    # @var target_m
    # Contains the position target of the currently set command in
    # the machine coordinate system.
    self.target_m = [None, None, None]

    # @var target_w
    # Contains the position target of the currently set command in
    # the currently selected coordinate system.
    self.target_w = [None, None, None]

    # @var offset
    # Contains the offset of the arc center from current position
    self.offset = [0, 0, 0]

    # @var radius
    # Contains the radius of the current arc
    self.radius = None

    # @var contains_radius
    # True if the current line contains a R word
    self.contains_radius = False

    # @var current_spindle_speed
    # Contains the current spindle speed (S word)
    self.current_spindle_speed = None

    # @var contains_spindle
    # True if the current line contains the S word
    self.contains_spindle = False

    # @var dist
    # Distance that current line will travel
    self.dist = 0

    # @var dists
    # Distance that current line will travel, in [x,y,z] directions
    self.dist_xyz = [0, 0, 0]

    # @var line_is_unsupported_cmd
    # True if current line is not in the whitelist
    self.line_is_unsupported_cmd = False

    self.comment = ""

    self.special_comment_prefix = "_gcm"

```

```

# precompile regular expressions
self._axes_regexps = []
self._axes_words = ["X", "Y", "Z"]
for i in range(0, 3):
    word = self._axes_words[i]
    self._axes_regexps.append(re.compile('.*' + word + r'([-.\d]+)'))

self._offset_regexps = []
self._offset_words = ["I", "J", "K"]
for i in range(0, 3):
    word = self._offset_words[i]
    self._offset_regexps.append(re.compile('.*' + word + r'([-.\d]+)'))

self._re_radius = re.compile(r'.*R([-.\d]+)')

self._re_use_var = re.compile(r'#(\d*)')
self._re_set_var = re.compile(r'^s#(\d+)([.\d-]+)')

self._re_feed = re.compile(r'.*F([-.\d]+)')
self._re_feed_replace = re.compile(r'F[.\d]+')

self._re_spindle = re.compile(r'.*S([.\d]+)')
self._re_spindle_replace = re.compile(r'S[.\d]+')

self._re_motion_mode = re.compile('G([0123])([^\d]|$)')
self._re_distance_mode = re.compile(r'(G9[01])([^\d]|$)')
self._re_plane_mode = re.compile(r'(G1[789])([^\d]|$)')
self._re_cs = re.compile(r'(G5[4-9])')

self._re_comment_paren_convert = re.compile(r'(\.*)((.*?))\s*$')
self._re_comment_paren_replace = re.compile(r'\(.*?\)')
self._re_comment_get_comment = re.compile(r'(\.*)((.*?))')

self._re_match_cmd_number = re.compile(r'([GMT])(\d+)')
self._re_expand_multicommands = re.compile(r'([GMT])')

# @var whitelist_commands
# Strip line from everything that is not in this list
self.whitelist_commands = {
    "G": [
        # non-modal commands
        4, # Dwell
        10, # set coordinate system
        28, # Go to predefined position
        30, # Go to predefined position
        53, # move in machine coordinates
        92, # set coordinate system offset

        # motion modes
        0, # fast linear
        1, # slow linear
    ]
}

```

2, # slow arc CW  
3, # slow arc CCW  
38, # probe  
80, # canned cycle stop

# feed rate modes

93, # inverse  
94, # normal

# units

20, # inch  
21, # mm

# distance modes

90, # absolute  
91, # incremental

# plane select

17, # XY  
18, # ZY  
19, # YZ

# tool length offset

43, # set compensation  
49, # cancel compensation

# coordinate systems

54,  
55,  
56,  
57,  
58,  
59,

],

"M": [

# program flow

0,  
1,  
2,  
30,

# coolant

8,  
9,

# spindle

3,  
4,  
5,

]

```
}
```

```
def reset(self):
```

```
    """
```

```
    Reset to initial state.
```

```
    """
```

```
    self.vars = {}
```

```
    self.current_feed = None
```

```
    self.current_motion_mode = 0
```

```
    self.current_distance_mode = "G90"
```

```
    self.callback("on_feed_change", self.current_feed)
```

```
@property
```

```
def position_m(self):
```

```
    return list(self.pos_m)
```

```
@position_m.setter
```

```
def position_m(self, pos):
```

```
    self.pos_m = list(pos)
```

```
    offset = self.cs_offsets[self.cs]
```

```
    self.pos_w = [
```

```
        self.pos_m[0] - offset[0],
```

```
        self.pos_m[1] - offset[1],
```

```
        self.pos_m[2] - offset[2]
```

```
    ]
```

```
@property
```

```
def current_cs(self):
```

```
    return self.cs
```

```
@current_cs.setter
```

```
def current_cs(self, label):
```

```
    self.cs = label
```

```
    offset = self.cs_offsets[self.cs]
```

```
    self.pos_w = [
```

```
        self.pos_m[0] - offset[0],
```

```
        self.pos_m[1] - offset[1],
```

```
        self.pos_m[2] - offset[2]
```

```
    ]
```

```
def set_line(self, line):
```

```
    """
```

```
    Load a Gcode line into the machine. It will be available in `self.line`
```

```
Parameters
```

```
-----
```

```
line : str
```

```
    A single G-Code line
```

```
    """
```

```
self.line = line
```

```
self.transform_comments()
```

```
def split_lines(self):
```

```
    """
```

Some Gcode generating software spit out 'composite' commands like "M3 T2". This really is bad practice.

This method splits such 'composite' commands up into separate commands and returns a list of commands. Machine state is not changed.

```
    """
```

```
    commands = re.sub(self._re_expand_multicommands,
                       r"\n\g<0>", self.line).strip()
```

```
    lines = commands.split("\n")
```

```
    lines[0] = lines[0] + self.comment # preserve comment
```

```
    return lines
```

```
def strip(self):
```

```
    """
```

Remove blank spaces and newlines from beginning and end, and remove blank spaces from the middle of the line.

```
    """
```

```
    self.line = self.line.replace(" ", "")
```

```
    self.line = self.line.strip()
```

```
def tidy(self):
```

```
    """
```

Strips G-Code not contained in the whitelist.

```
    """
```

```
# transform [MG]\d to G\d\d for better parsing
```

```
def format_cmd_number(matchobj):
```

```
    cmd = matchobj.group(1)
```

```
    cmd_nr = int(matchobj.group(2))
```

```
    self.line_is_unsupported_cmd = not (
```

```
        cmd in self.whitelist_commands
```

```
        and cmd_nr in self.whitelist_commands[cmd]
```

```
    )
```

```
    return '{}{:02d}'.format(cmd, cmd_nr)
```

```
self.line = re.sub(self._re_match_cmd_number,
```

```
                  format_cmd_number, self.line)
```

```
if self.line_is_unsupported_cmd:
```

```
    self.line = ';' + self.line + \
```

```
        ';' + self.special_comment_prefix + \
```

```
        '.unsupported'
```

```
def parse_state(self):
```

```
    """
```

This method...

- \* parses motion mode
- \* parses distance mode
- \* parses plane mode
- \* parses feed rate
- \* parses spindle speed
- \* parses arcs (offsets and radii)
- \* calculates travel distances

... and updates the machine's state accordingly.

"""

# parse G0 .. G3 and remember

m = re.match(self.\_re\_motion\_mode, self.line)

if m:

self.current\_motion\_mode = int(m.group(1))

# parse G90 and G91 and remember

m = re.match(self.\_re\_distance\_mode, self.line)

if m:

self.current\_distance\_mode = m.group(1)

# parse G17, G18 and G19 and remember

m = re.match(self.\_re\_plane\_mode, self.line)

if m:

self.current\_plane\_mode = m.group(1)

m = re.match(self.\_re\_cs, self.line)

if m:

self.current\_cs = m.group(1)

# see if current line has F

m = re.match(self.\_re\_feed, self.line)

self.contains\_feed = True if m else False

if m:

self.feed\_in\_current\_line = float(m.group(1))

# look for spindle S

m = re.match(self.\_re\_spindle, self.line)

self.contains\_spindle = True if m else False

if m:

self.current\_spindle\_speed = int(m.group(1))

# arc parsing and calculations

if self.current\_motion\_mode == 2 or self.current\_motion\_mode == 3:

self.offset = [None, None, None]

for i in range(0, 3):

# loop over I, J, K offsets

regex = self.\_offset\_regexps[i]

m = re.match(regex, self.line)

```

        if m:
            self.offset[i] = float(m.group(1))

# parses arcs
m = re.match(self._re_radius, self.line)
self.contains_radius = True if m else False
if m:
    self.radius = float(m.group(1))

# calculate distance traveled by this G-Code cmd in xyz
self.dist_xyz = [0, 0, 0]
for i in range(0, 3):
    # loop over X, Y, Z axes
    regexp = self._axes_regexp[i]

    m = re.match(regexp, self.line)
    if m:
        if self.current_distance_mode == "G90":
            # absolute distances
            self.target_m[i] = self.cs_offsets[self.cs][i] + \
                float(m.group(1))
            self.target_w[i] = float(m.group(1))

            # calculate distance
            self.dist_xyz[i] = self.target_m[i] - self.pos_m[i]
        else:
            # G91 relative distances
            self.dist_xyz[i] = float(m.group(1))
            self.target_m[i] += self.dist_xyz[i]
            self.target_w[i] += self.dist_xyz[i]
    else:
        # no movement along this axis, stays the same
        self.target_m[i] = self.pos_m[i]
        self.target_w[i] = self.pos_w[i]

# calculate travelling distance
self.dist = math.sqrt(
    self.dist_xyz[0] * self.dist_xyz[0] +
    self.dist_xyz[1] * self.dist_xyz[1] +
    self.dist_xyz[2] * self.dist_xyz[2]
)

```

```
def fractionize(self):
```

```
    """
```

Breaks lines longer than a certain threshold into shorter segments.

Also breaks circles into segments.

Returns a list of command strings. Does not update the machine state.

```
    """
```

```
result = []
```

```
if (self.do_fractionize_lines
    and self.current_motion_mode == 1
    and self.dist > self.fract_linear_threshold):
    result = self.__fractionize_linear_motion()
```

```
elif (self.do_fractionize_arcs and
      (self.current_motion_mode == 2 or self.current_motion_mode == 3)
      ):
    result = self.__fractionize_circular_motion()
```

```
else:
    # this motion cannot be fractionized
    # return the line as it was passed in
    result = [self.line]
```

```
result[0] = result[0] + self.comment # preserve comment
return result
```

```
def done(self):
```

```
    """
```

```
    Call this method when all processing/inspecting of a command
    has been done.
```

```
    This will virtually 'move' the tool of the machine
    if the current command is a motion command.
```

```
    """
```

```
if not (self.current_motion_mode == 0 or self.current_motion_mode == 1):
    # only G0 and G1 can stay active without re-specifying
    self.current_motion_mode = None
```

```
# move the 'tool'
```

```
for i in range(0, 3):
```

```
    # loop over X, Y, Z axes
```

```
    if self.target_m[i] is not None: # keep state
```

```
        self.pos_m[i] = self.target_m[i]
```

```
        self.pos_w[i] = self.target_w[i]
```

```
# re-add comment
```

```
self.line += self.comment
```

```
def find_vars(self):
```

```
    """
```

```
    Parses all variables in a G-Code line (#1, #2, etc.) and populates
    the internal `vars` dict with corresponding keys and values
```

```
    """
```

```
m = re.match(self._re_set_var, self.line)
```

```
if m:
```

```
    key = m.group(1)
```

```
    val = str(float(m.group(2))) # get rid of extra zeros
```

```
self.vars[key] = val
self.line = ";" + self.line
```

```
# find variable usages
keys = re.findall(self._re_use_var, self.line)
for key in keys:
    if key not in self.vars:
        self.vars[key] = None
```

```
def substitute_vars(self):
    """
    Substitute a variable with a value from the `vars` dict.
    """
    keys = re.findall(self._re_use_var, self.line)

    for key in keys:
        val = None
        if key in self.vars:
            val = self.vars[key]

        if val is None:
            self.line = ""
            self.callback("on_var_undefined", key)
            return self.line
        else:
            self.line = self.line.replace("#" + key, str(val))
            self.logger.info("substituted variable #{0} -> {1}".format(key, val))
```

```
def scale_spindle(self):
    if self.contains_spindle:
        # strip the original S setting
        self.line = re.sub(self._re_spindle_replace, "", self.line).strip()
        self.line += "S{:d}".format(
            int(self.current_spindle_speed * self.spindle_factor))
```

```
def override_feed(self):
    """
    Call this method to

    * get a callback when the current command contains an F word
    *
    """
```

```
if not self.do_feed_override and self.contains_feed:
    # Notify caller of detected feed in current line (useful for UIs)
    if self.current_feed != self.feed_in_current_line:
        self.callback("on_feed_change", self.feed_in_current_line)
        self.current_feed = self.feed_in_current_line
```

```
if self.do_feed_override and self.request_feed:
```

```

if self.contains_feed:
    # strip the original F setting
    self.logger.info("stripping feed " + self.line)
    self.line = re.sub(self._re_feed_replace,
                        "", self.line).strip()

if (self.current_feed != self.request_feed):
    self.line += "F{:0.1f}".format(self.request_feed)
    self.current_feed = self.request_feed
    self.logger.info("overriding feed " + str(self.current_feed))
    self.callback("on_feed_change", self.current_feed)

# TODO: This method can be made 'private' as it is called from `set_line()`
def transform_comments(self):
    """
    Comments in Gcode can be set with semicolon or parentheses.
    This method transforms parentheses comments to semicolon comments.
    """

    # transform () comment at end of line into semicolon comment
    self.line = re.sub(self._re_comment_paren_convert,
                        r"\g<1>;\g<2>", self.line)

    # remove all in-line () comments
    self.line = re.sub(self._re_comment_paren_replace, "", self.line)

    m = re.match(self._re_comment_get_comment, self.line)
    if m:
        self.line = m.group(1)
        self.comment = m.group(2)
    else:
        self.comment = ""

# 'private' methods

def __fractionize_circular_motion(self):
    """
    This function is a direct port of Grbl's C code into Python (gcode.c)
    with slight refactoring for Python by Michael Franzl.
    See https://github.com/grbl/grbl
    """

    # implies self.current_motion_mode == 2 or self.current_motion_mode == 3

    if self.current_plane_mode == 'G17':
        axis_0 = 0 # X axis
        axis_1 = 1 # Y axis
        axis_linear = 2 # Z axis
    elif self.current_plane_mode == 'G18':
        axis_0 = 2 # Z axis
        axis_1 = 0 # X axis

```

```

axis_linear = 1 # Y axis
elif self.current_plane_mode == 'G19':
    axis_0 = 1 # Y axis
    axis_1 = 2 # Z axis
    axis_linear = 0 # X axis

is_clockwise_arc = True if self.current_motion_mode == 2 else False

# deltas between target and (current) position
x = self.target_w[axis_0] - self.pos_w[axis_0]
y = self.target_w[axis_1] - self.pos_w[axis_1]

if self.contains_radius:
    # RADIUS MODE
    # R given, no IJK given, self.offset must be calculated

    if tuple(self.target_w) == tuple(self.pos_w):
        self.logger.error(
            "Arc in Radius Mode: Identical start/end {}".format(self.line))
        return [self.line]

    h_x2_div_d = 4.0 * self.radius * self.radius - x * x - y * y

    if h_x2_div_d < 0:
        self.logger.error(
            "Arc in Radius Mode: Radius error {}".format(self.line))
        return [self.line]

    # Finish computing h_x2_div_d.
    h_x2_div_d = -math.sqrt(h_x2_div_d) / math.sqrt(x * x + y * y)

    if not is_clockwise_arc:
        h_x2_div_d = -h_x2_div_d

    if self.radius < 0:
        h_x2_div_d = -h_x2_div_d
        self.radius = -self.radius

    self.offset[axis_0] = 0.5*(x-(y*h_x2_div_d))
    self.offset[axis_1] = 0.5*(y+(x*h_x2_div_d))

else:
    # CENTER OFFSET MODE, no R given so must be calculated

    if self.offset[axis_0] is None or self.offset[axis_1] is None:
        raise Exception(
            "Arc in Offset Mode: No offsets in plane. {}".format(self.line))
        # self.logger.error("Arc in Offset Mode: No offsets in plane")
        # return [self.line]

    # Arc radius from center to target

```

```

x -= self.offset[axis_0]
y -= self.offset[axis_1]
target_r = math.sqrt(x * x + y * y)

# Compute arc radius for mc_arc.
# Defined from current location to center.
self.radius = math.sqrt(
    self.offset[axis_0] * self.offset[axis_0] +
    self.offset[axis_1] * self.offset[axis_1]
)

# Compute difference between current location and target radii for final error-checks.
delta_r = math.fabs(target_r - self.radius)
if delta_r > 0.005:
    if delta_r > 0.5:
        raise Exception("Arc in Offset Mode: Invalid Target. r={:f} delta_r={:f} {}".format(
            self.radius, delta_r, self.line))

    if delta_r > (0.001 * self.radius):
        raise Exception("Arc in Offset Mode: Invalid Target. r={:f} delta_r={:f} {}".format(
            self.radius, delta_r, self.line))

```

```

gcode_list = self.__mc_arc(
    self.pos_w,
    self.target_w,
    self.offset,
    self.radius,
    axis_0,
    axis_1,
    axis_linear,
    is_clockwise_arc
)
return gcode_list

```

```

def __mc_arc(
    self,
    position,
    target,
    offset,
    radius,
    axis_0,
    axis_1,
    axis_linear,
    is_clockwise_arc
):
    """

```

This function is a direct port of Grbl's C code into Python (motion\_control.c) with slight refactoring for Python by Michael Franzl.  
 See <https://github.com/grbl/grbl>

```

gcode_list = []
gcode_list.append("; " + self.special_comment_prefix +
    ".arc_begin[{}]" .format(self.line))

do_restore_distance_mode = False
if self.current_distance_mode == "G91":
    # it's bad to concatenate many small floating point segments due to accumulating errors
    # each arc will use G90
    do_restore_distance_mode = True
    gcode_list.append("G90")

center_axis0 = position[axis_0] + offset[axis_0]
center_axis1 = position[axis_1] + offset[axis_1]
# radius vector from center to current location
r_axis0 = -offset[axis_0]
r_axis1 = -offset[axis_1]
# radius vector from target to center
rt_axis0 = target[axis_0] - center_axis0
rt_axis1 = target[axis_1] - center_axis1

angular_travel = math.atan2(
    r_axis0 * rt_axis1 - r_axis1 * rt_axis0,
    r_axis0 * rt_axis0 + r_axis1 * rt_axis1
)

arc_tolerance = 0.004
arc_angular_travel_epsilon = 0.0000005

if is_clockwise_arc: # Correct atan2 output per direction
    if angular_travel >= -arc_angular_travel_epsilon:
        angular_travel -= 2*math.pi
    else:
        if angular_travel <= arc_angular_travel_epsilon:
            angular_travel += 2*math.pi

segments = math.floor(
    math.fabs(0.5 * angular_travel * radius) /
    math.sqrt(arc_tolerance * (2 * radius - arc_tolerance))
)

words = ["X", "Y", "Z"]
if segments:
    theta_per_segment = angular_travel / segments
    linear_per_segment = (
        target[axis_linear] - position[axis_linear]) / segments

    position_last = list(position)
    for i in range(1, segments):
        cos_Ti = math.cos(i * theta_per_segment)
        sin_Ti = math.sin(i * theta_per_segment)
        r_axis0 = -offset[axis_0] * cos_Ti + offset[axis_1] * sin_Ti

```

```

r_axis1 = -offset[axis_0] * sin_Ti - offset[axis_1] * cos_Ti

position[axis_0] = center_axis0 + r_axis0
position[axis_1] = center_axis1 + r_axis1
position[axis_linear] += linear_per_segment

gcodeline = ""
if i == 1:
    gcodeline += "G1"

for a in range(0, 3):
    if position[a] != position_last[a]: # only write changes
        txt = "{}{:0.3f}".format(words[a], position[a])
        txt = txt.rstrip("0").rstrip(".")
        gcodeline += txt
        position_last[a] = position[a]

if i == 1:
    if self.contains_feed:
        gcodeline += "F{:1f}".format(
            self.feed_in_current_line)
    if self.contains_spindle:
        gcodeline += "S{:d}".format(self.current_spindle_speed)

gcode_list.append(gcodeline)

# make sure we arrive at target
gcodeline = ""
if segments <= 1:
    gcodeline += "G1"

for a in range(0, 3):
    if target[a] != position[a]:
        txt = "{}{:0.3f}".format(words[a], target[a])
        txt = txt.rstrip("0").rstrip(".")
        gcodeline += txt

if segments <= 1:
    # no segments were rendered (very small arc)
    # so we have to put S and F here
    if self.contains_feed:
        gcodeline += "F{:1f}".format(self.feed_in_current_line)
    if self.contains_spindle:
        gcodeline += "S{:d}".format(self.current_spindle_speed)

gcode_list.append(gcodeline)

if do_restore_distance_mode:
    gcode_list.append(self.current_distance_mode)

gcode_list.append("; " + self.special_comment_prefix + ".arc_end")

```

```

return gcode_list

def __fractionize_linear_motion(self):
    gcode_list = []
    gcode_list.append("; " + self.special_comment_prefix +
        ".line_begin[{}].format(self.line))

    num_fractions = int(self.dist / self.fract_linear_segment_len)

    for k in range(0, num_fractions):
        # render segments
        txt = ""
        if k == 0:
            txt += "G1"

        for i in range(0, 3):
            # loop over X, Y, Z axes
            segment_length = self.dist_xyz[i] / num_fractions
            coord_rel = (k + 1) * segment_length
            if self.current_distance_mode == "G90":
                # absolute distances
                coord_abs = self.pos_w[i] + coord_rel
                if coord_rel != 0:
                    # only output for changes
                    txt += "{}{:0.3f}".format(
                        self._axes_words[i], coord_abs)
                    txt = txt.rstrip("0").rstrip(".")
            else:
                # relative distances
                txt += "{}{:0.3f}".format(
                    self._axes_words[i], segment_length)
                txt = txt.rstrip("0").rstrip(".")

        if k == 0:
            if self.contains_feed:
                txt += "F{:0.1f}".format(self.feed_in_current_line)
            if self.contains_spindle:
                txt += "S{:d}".format(self.current_spindle_speed)

        gcode_list.append(txt)

    gcode_list.append("; " + self.special_comment_prefix + ".line_end")
    return gcode_list

def __default_callback(self, status, *args):
    self.logger.info("default callback", status, args)

```

**File: callbackloghandler.py****Directory: C:/Users/sense/PycharmProjects/cnc\_aoi/.venv/Lib/site-packages/grbl\_streamer**

"""

grbl-streamer - Universal interface module for the grbl CNC firmware  
Copyright (C) 2015 Michael Franzl

This file is part of grbl-streamer.

grbl-streamer is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

grbl-streamer is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with pygltpainter. If not, see <<https://www.gnu.org/licenses/>>.

"""

```
import logging
```

```
class CallbackLogHandler(logging.StreamHandler):
```

```
    def __init__(self, cb=None):
```

```
        super(CallbackLogHandler, self).__init__()
```

```
        self.callback = cb
```

```
    def emit(self, record):
```

```
        if self.callback:
```

```
            #txt = "{0} {1}".format(record.level, record.msg)
```

```
            #print(self.format(record))
```

```
            #self.callback("on_log", self.format(record))
```

```
            #self.callback("on_log", record.levelno, record.msg % record.args)
```

```
            self.callback('on_log', record)
```

```
        else:
```

```
            logging.StreamHandler.emit(self, record)
```

**File: interface.py****Directory: C:/Users/sense/PycharmProjects/cnc\_aoi/.venv/Lib/site-packages/grbl\_streamer**

"""

grbl-streamer - Universal interface module for the grbl CNC firmware  
Copyright (C) 2015 Michael Franzl

This file is part of grbl-streamer.

grbl-streamer is free software: you can redistribute it and/or modify

it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

grbl-streamer is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with pyglpainter. If not, see <<https://www.gnu.org/licenses/>>.

"""

```
import serial
import threading
import logging
```

```
class Interface:
```

```
    """Implements opening, closing, writing and threaded reading
    from the serial port. Read data are put into a Thread Queue.
    """
```

```
    def __init__(self, name, path, baud=115200):
```

```
        """Straightforward initialization tasks.
```

```
        @param name
```

```
        An informal name of the instance. Useful if you are running
        several instances to control several serial ports at once.
        It is only used for logging output and UI messages.
```

```
        @param path
```

```
        The serial port device node living under /dev.
        e.g. /dev/ttyACM0 or /dev/ttyUSB0
```

```
        @param baud
```

```
        The baud rate. Default is 115200 for Grbl > v0.9i.
```

```
        """
```

```
        self.name = name
```

```
        self.path = path
```

```
        self.baud = baud
```

```
        self.queue = None
```

```
        self.logger = logging.getLogger('GrblStreamer.interface')
```

```
        self._buf_receive = "
```

```
        self._do_receive = False
```

```
    def start(self, queue):
```

```
        """
```

```
        Open the device node and start a Thread for reading.
```

@param queue

An instance of Python3's `Queue()` class.

"""

self.queue = queue

self.logger.info(

'%s: connecting to %s with baudrate %i',

self.name,

self.path,

self.baud)

self.serialport = serial.Serial(

self.path,

self.baud,

parity=serial.PARITY\_NONE,

stopbits=serial.STOPBITS\_ONE,

bytesize=serial.EIGHTBITS,

timeout=1,

writeTimeout=0,

)

self.serialport.flushInput()

self.serialport.flushOutput()

self.\_do\_receive = True

self.serial\_thread = threading.Thread(target=self.\_receiving)

self.serial\_thread.start()

def stop(self):

"""

Close the device node and shut down the reading Thread.

"""

self.\_do\_receive = False

self.logger.info('%s: stop()', self.name)

self.serial\_thread.join()

self.logger.info('%s: JOINED thread', self.name)

self.logger.info('%s: Closing port', self.name)

self.serialport.flushInput()

self.serialport.flushOutput()

self.serialport.close()

def write(self, data):

"""

Write `data` to the device node.

If data is empty, no write is performed.

The number of written characters is returned.

"""

if len(data) > 0:

num\_written = self.serialport.write(bytes(data, 'ascii'))

return num\_written

else:

self.logger.debug('%s: nothing to write', self.name)

```

def _receiving(self):
    while self._do_receive:
        data = self.serialport.read(1)
        waiting = self.serialport.inWaiting()
        data += self.serialport.read(waiting)
        self._handle_data(data)

def _handle_data(self, data):
    try:
        asci = data.decode('ascii')
    except UnicodeDecodeError:
        self.logger.info(
            '%s: Received a non-ascii byte. Probably junk. Dropping it.',
            self.name)
        asci = ""

    for i in range(0, len(asci)):
        char = asci[i]
        self._buf_receive += char
        # not all received lines are complete (end with \n)
        if char == "\n":
            self.queue.put(self._buf_receive.strip())
            self._buf_receive = ""

```

**File: \_\_init\_\_.py**

**Directory: C:/Users/sense/PycharmProjects/cnc\_aoi/.venv/Lib/site-packages/grbl\_streamer**

"""

grbl-streamer - Universal interface module for the grbl CNC firmware  
 Copyright (C) 2015 Michael Franzl

This file is part of grbl-streamer.

grbl-streamer is free software: you can redistribute it and/or modify  
 it under the terms of the GNU General Public License as published by  
 the Free Software Foundation, either version 3 of the License, or  
 (at your option) any later version.

grbl-streamer is distributed in the hope that it will be useful,  
 but WITHOUT ANY WARRANTY; without even the implied warranty of  
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the  
 GNU General Public License for more details.

You should have received a copy of the GNU General Public License  
 along with pygltpainter. If not, see <<https://www.gnu.org/licenses/>>.

"""

```
import logging
import time
import re
import threading
import atexit
```

```
from queue import Queue
```

```
from .interface import Interface
from .callbackloghandler import CallbackLogHandler
```

```
from gcode_machine import GcodeMachine
```

```
class GrblStreamer:
```

```
    """ A universal Grbl CNC firmware interface module for Python3
    providing a convenient high-level API for scripting or integration
    into parent applications like GUI's.
```

There are a number of streaming applications available for the Grbl CNC controller, but none of them seem to be an universal, re-usable standard Python module. GrblStreamer attempts to fill that gap.

See README for usage examples.

Features:

- \* Re-usable across projects
- \* Non-blocking
- \* Asynchronous (event-based) callbacks for the parent application
- \* Two streaming modes: Incremental or fast ("counting characters")
- \* Defined shutdown
- \* G-Code cleanup
- \* G-Code variable expansion
- \* Dynamic feed override
- \* Buffer stashing
- \* Job halt and resume

Callbacks:

After assigning your own callback function (callback = ...) you will receive the following signals:

on\_boot  
: Emitted whenever Grbl boots (e.g. after a soft reset).  
: No arguments.

on\_read  
: Emitted whenever a line is read by GrblStreamer  
: 1 argument: line

on\_disconnected

: Emitted whenever the serial port has been closed.

: No arguments

on\_log

: Emitted for informal logging or debugging messages.

: 1 argument: LogRecord instance

on\_line\_sent

: Emitted whenever a line is actually sent to Grbl.

: 2 arguments: job\_line\_number, line

on\_bufsize\_change

: Emitted whenever lines have been appended to the buffer

: 1 argument: linecount

on\_line\_number\_change

: Emitted whenever the current buffer position has been changed

: 1 argument: line\_number

on\_processed\_command

: Emitted whenever Grbl confirms a command with "ok" and is now being executed physically

: 2 arguments: processed line number, processed line

on\_alarm

: Emitted whenever Grbl sends an "ALARM" line

: 1 argument: the full line Grbl sent

on\_error

: Emitted whenever Grbl sends an "ERROR" line

: 3 arguments: the full line Grbl sent, the line that caused the error, the line number in the buffer that caused the error

on\_rx\_buffer\_percent

: Reports Grbl's serial receive buffer fill in percent. Emitted frequently while streaming.

: 1 argument: percentage integer from 0 to 100

on\_progress\_percent

: Reports the completion of the current job/buffer in percent. Emitted frequently while streaming.

: 1 argument: percentage integer from 0 to 100

on\_job\_completed

: Emitted when the current job/buffer has been streamed and physically executed entirely

on\_stateupdate

: Emitted whenever Grbl's state has changed

: 3 arguments: Grbl's mode ('Idle', 'Run' etc.), machine position tuple, working position tuple

on\_hash\_stateupdate

: Emitted after Grbl's 'hash' EEPROM settings (`\$#`) have been received

: 1 argument: dict of the settings

on\_settings\_downloaded

: Emitted after Grbl's EEPROM settings (`\$\$`) have been received

: 1 argument: dict of the settings

on\_gcode\_parser\_stateupdate

: Emitted after Grbl's G-Code parser state has been received

: 1 argument: list of the state variables

on\_simulation\_finished

: Emitted when GrblStreamer's target is set to "simulator" and the job is executed.

: 1 argument: list of all G-Code commands that would have been sent to Grbl

on\_vars\_change

: Emitted after G-Code is loaded into the buffer and variables have been detected

: 1 argument: a dict of the detected variables

on\_preprocessor\_feed\_change

: Emitted when a F keyword is parsed from the G-Code.

: 1 argument: the feed rate in mm/min

"""

\_\_version\_\_ = "0.5.0"

def \_\_init\_\_(self, callback, name="mygrbl"):

"""Straightforward initialization tasks.

@param callback

Set your own function that will be called when a number of asynchronous events happen. Useful for UI's. The default function will just log to stdout.

This callback function will receive two arguments. The first is a string giving a label of the event, and the second is a variable argument list `\*args` containing data pertaining to the event.

Note that this function may be called from a Thread.

@param name

An informal name of the instance. Useful if you are running several instances to control several CNC machines at once. It is only used for logging output and UI messages.

"""

# @var name

# Set an informal name of the instance. Useful if you are running several instances to control several CNC machines at once. It is only used for logging output and UI messages.  
 self.name = name

# @var cmode

# Get Grbl's current mode.

```

# Will be strings 'Idle', 'Check', 'Run'
self.cmode = None

# @var cmpos
# Get a 3-tuple containing the current coordinates relative
# to the machine origin.
self.cmpos = (0, 0, 0)

# @var cwpos
# Get a 3-tuple containing the current working coordinates.
# Working coordinates are relative to the currently selected
# coordinate system.
self.cwpos = (0, 0, 0)

# @var gps
# Get list of 12 elements containing the 12 Gcode Parser State
# variables of Grbl which are obtained by sending the raw
# command `$G`. Will be available after setting
# `hash_state_requested` to True.
self.gps = [
    '0', # motion mode
    '54', # current coordinate system
    '17', # current plane mode
    '21', # units
    '90', # current distance mode
    '94', # feed rate mode
    '0', # program mode
    '0', # spindle state
    '5', # coolant state
    '0', # tool number
    '99', # current feed
    '0', # spindle speed
]

# @var poll_interval
# Set an interval in seconds for polling Grbl's state via
# the `?` command. The Grbl Wiki recommends to set this no lower
# than 0.2 (5 per second).
self.poll_interval = 0.2

# @var settings
# Get a dictionary of Grbl's EEPROM settings which can be read
# after sending the `$$` command, or more conveniently after
# calling the method `request_settings()` of this class.
self.settings = {
    130: {'val': '1000', 'cmt': 'width'},
    131: {'val': '1000', 'cmt': 'height'}
}

# @var settings_hash
# Get a dictionary of Grbl's 'hash' settings (also stored in the

```

```
# EEPROM) which can be read after sending the `$$` command. It
# contains things like coordinate system offsets. See Grbl
# documentation for more info. Will be available shortly after
# setting `hash_state_requested` to `True`.
```

```
self.settings_hash = {
```

```
    'G54': (0, 0, 0),
```

```
    'G55': (0, 0, 0),
```

```
    'G56': (0, 0, 0),
```

```
    'G57': (0, 0, 0),
```

```
    'G58': (0, 0, 0),
```

```
    'G59': (0, 0, 0),
```

```
    'G28': (0, 0, 0),
```

```
    'G30': (0, 0, 0),
```

```
    'G92': (0, 0, 0),
```

```
    'TLO': 0,
```

```
    'PRB': (0, 0, 0),
```

```
}
```

```
# @var gcode_parser_state_requested
```

```
# Set this variable to `True` to receive a callback with the
# event string "on_gcode_parser_stateupdate" containing
# data that Grbl sends after receiving the `$G` command.
```

```
# After the callback, this variable reverts to `False`.
```

```
self.gcode_parser_state_requested = False
```

```
# @var hash_state_requested
```

```
# Set this variable to `True` to receive a callback with the
# event string "on_hash_stateupdate" containing
# the requested data. After the callback, this variable reverts
# to `False`.
```

```
self.hash_state_requested = False
```

```
# @var logger
```

```
# The logger used by this class. The default is Python's own
# logger module. Use `setup_logging()` to attach custom
# log handlers.
```

```
self.logger = logging.getLogger('GrblStreamer')
```

```
self.logger.setLevel(logging.DEBUG)
```

```
self.logger.propagate = False
```

```
# @var target
```

```
# Set this to change the output target. Default is "firmware"
# which means the serial port. Another target is "simulator",
# you will receive a callback with even string
# "on_simulation_finished" and a buffer of the G-Code commands
# that would have been sent out to Grbl.
```

```
# TODO: Add "file" target.
```

```
self.target = 'firmware'
```

```
# @var connected
```

```
# `True` when connected to Grbl (after boot), otherwise `False`
```

```
self.connected = False
```

```
# @var preprocessor  
# All G-code commands will go through the preprocessor  
# before they are sent out via the serial port. The preprocessor  
# keeps track of, and can dynamically change, feed rates, as well  
# as substitute variables. It has its own state and callback  
# functions.  
self.preprocessor = GcodeMachine()  
self.preprocessor.callback = self._preprocessor_callback
```

```
# @var travel_dist_buffer  
# The total distance of all G-Codes in the buffer.  
self.travel_dist_buffer = {}
```

```
# @var travel_dist_current  
# The currently travelled distance. Can be used to calculate ETA.  
self.travel_dist_current = {}
```

```
# @var is_standstill  
# If the machine is currently not moving  
self.is_standstill = False
```

```
self._ifacepath = None  
self._last_setting_number = 132
```

```
self._last_cmode = None  
self._last_cmpos = (0, 0, 0)  
self._last_cwpos = (0, 0, 0)
```

```
self._standstill_watchdog_increment = 0
```

```
self._rx_buffer_size = 128  
self._rx_buffer_fill = []  
self._rx_buffer_backlog = []  
self._rx_buffer_backlog_line_number = []  
self._rx_buffer_fill_percent = 0
```

```
self._current_line = "  
self._current_line_sent = True  
self._streaming_mode = None  
self._wait_empty_buffer = False  
self.streaming_complete = True  
self.job_finished = True  
self._streaming_src_end_reached = True  
self._streaming_enabled = True  
self._error = False  
self._incremental_streaming = False
```

```
self.buffer = []  
self.buffer_size = 0
```

```
self._current_line_nr = 0
```

```
self.buffer_stash = []
```

```
self.buffer_size_stash = 0
```

```
self._current_line_nr_stash = 0
```

```
self._poll_keep_alive = False
```

```
self._iface_read_do = False
```

```
self._thread_polling = None
```

```
self._thread_read_iface = None
```

```
self._iface = None
```

```
self._queue = Queue()
```

```
self._loghandler = None
```

```
# general-purpose counter for timing tasks inside of _poll_state
```

```
self._counter = 0
```

```
self._callback = callback
```

```
atexit.register(self.disconnect)
```

```
# supply defaults to GUI to make it operational
```

```
self._callback('on_settings_downloaded', self.settings)
```

```
self._callback('on_hash_stateupdate', self.settings_hash)
```

```
self.preprocessor.cs_offsets = self.settings_hash
```

```
self._callback('on_gcode_parser_stateupdate', self.gps)
```

```
def setup_logging(self, handler=None):
```

```
    """Assign a custom log handler.
```

GrblStreamer can be used in both console applications as well as integrated in other projects like GUI's. Therefore, logging to stdout is not always useful. You can pass a custom log message handler to this method. If no handler is passed in, the default handler is an instance of class `CallbackLogHandler` (see file `callback\_loghandler.py` included in this module). CallbackLogHandler will deliver logged strings as callbacks to the parent application, the event string will be "on\_log".

```
@param handler=None
```

```
An instance of a subclass inheriting from `logging.StreamHandler`
```

```
"""
```

```
# The default log handler shipped with this module will call
```

```
# self._callback() with first parameter "on_log" and second
```

```
# parameter with the logged string.
```

```
if handler is not None:
```

```
    self._loghandler = handler
```

else:

```
lh = CallbackLogHandler()
lh.callback = self._callback
self._loghandler = lh
```

```
self.logger.addHandler(self._loghandler)
```

```
def cnect(self, path=None, baudrate=115200):
```

```
    """
```

Connect to the RS232 port of the Grbl controller.

@param path=None Path to the device node

This is done by instantiating a RS232 class, included in this module, which by itself block-listens (in a thread) to asynchronous data sent by the Grbl controller.

```
    """
```

```
if path is None or path.strip() == "":
```

```
    return
```

```
else:
```

```
    self._ifacepath = path
```

```
if self._iface is None:
```

```
    self.logger.debug('{}: Setting up interface on {}'.format(self.name, self._ifacepath))
```

```
    self._iface = Interface('iface_' + self.name, self._ifacepath, baudrate)
```

```
    self._iface.start(self._queue)
```

```
else:
```

```
    self.logger.info('{}: Cannot start another interface. There is already an interface {}'.format(self.name, self._iface))
```

```
self._iface_read_do = True
```

```
self._thread_read_iface = threading.Thread(target=self._onread)
```

```
self._thread_read_iface.start()
```

```
self.softreset()
```

```
def disconnect(self):
```

```
    """
```

This method provides a controlled shutdown and cleanup of this module.

It stops all threads, joins them, then closes the serial connection. For a safe shutdown of Grbl you may also want to call `softreset()` before you call this method.

```
    """
```

```
if not self.is_connected():
```

```
    return
```

```
self.poll_stop()
```

```
self._iface.stop()
```

```
self._iface = None
```

```
self.logger.debug('{}: Please wait until reading thread has joined...'.format(self.name))
self._iface_read_do = False
# The thread will not join without putting a last queue message:
self._queue.put("dummy_msg_for_joining_thread")
self._thread_read_iface.join()
self.logger.debug('{}: Reading thread successfully joined.'.format(self.name))
```

```
self.connected = False
```

```
self._callback('on_disconnected')
```

```
def softreset(self):
```

```
    """
```

```
    Immediately sends `Ctrl-X` to Grbl.
```

```
    """
```

```
    self._iface.write('\x18') # Ctrl-X
```

```
    self.update_preprocessor_position()
```

```
def abort(self):
```

```
    """
```

```
    An alias for `softreset()`.
```

```
    """
```

```
    if not self.is_connected():
```

```
        return
```

```
    self.softreset()
```

```
def hold(self):
```

```
    """
```

```
    Immediately sends the feed hold command (exclamation mark)
    to Grbl.
```

```
    """
```

```
    if not self.is_connected():
```

```
        return
```

```
    self._iface.write('!')
```

```
def resume(self):
```

```
    """
```

```
    Immediately send the resume command (tilde) to Grbl.
```

```
    """
```

```
    if not self.is_connected():
```

```
        return
```

```
    self._iface.write('~')
```

```
def killalarm(self):
```

```
    """
```

```
    Immediately send the kill alarm command ($X) to Grbl.
```

```
    """
```

```
    self._iface.write('$X\n')
```

```
def homing(self):
```

```
''''
```

Immediately send the homing command (\$H) to Grbl.

```
''''
```

```
self._iface_write('$H\n')
```

```
def poll_start(self):
```

```
''''
```

Starts forever polling Grbl's status with the `?` command. The polling interval is controlled by setting `self.poll\_interval`.

You will receive callbacks with the "on\_stateupdate" event string containing 3 data parameters self.cmode, self.cmpos, self.cwpos, but only when Grbl's state CHANGES.

```
''''
```

```
if not self.is_connected():
```

```
    return
```

```
self._poll_keep_alive = True
```

```
self._last_cmode = None
```

```
if self._thread_polling is None:
```

```
    self._thread_polling = threading.Thread(target=self._poll_state)
```

```
    self._thread_polling.start()
```

```
    self.logger.debug('{}: Polling thread started'.format(self.name))
```

```
else:
```

```
    self.logger.debug('{}: Polling thread already running...'.format(self.name))
```

```
def poll_stop(self):
```

```
''''
```

Stops polling that has been started with `poll\_start()`

```
''''
```

```
if not self.is_connected():
```

```
    return
```

```
if self._thread_polling is not None:
```

```
    self._poll_keep_alive = False
```

```
    self.logger.debug('{}: Please wait until polling thread has joined...'.format(self.name))
```

```
    self._thread_polling.join()
```

```
    self.logger.debug('{}: Polling thread has successfully joined...'.format(self.name))
```

```
else:
```

```
    self.logger.debug('{}: Cannot stop polling thread because none running.'.format(self.name))
```

```
self._thread_polling = None
```

```
def set_feed_override(self, val):
```

```
''''
```

Enable or disable the feed override feature.

@param val

Pass `True` or `False` as argument to enable or disable dynamic feed override. After passing `True`, you may set the requested feed by calling `self.request\_feed()` one or many times.

```

"""
self.preprocessor.do_feed_override = val

def request_feed(self, requested_feed):
    """
    Override the feed speed.
    Effective only when you set `set_feed_override(True)`.

    @param requested_feed
    The feed speed in mm/min.
    """
    self.preprocessor.request_feed = float(requested_feed)

@property
def incremental_streaming(self):
    return self._incremental_streaming

@incremental_streaming.setter
def incremental_streaming(self, value):
    """
    Incremental streaming means that a new command is sent to Grbl
    only after Grbl has responded with 'ok' to the last sent
    command. This is necessary to flash $ settings to the EEPROM.

    Non-incremental streaming means that Grbl's 100-some-byte
    receive buffer will be kept as full as possible at all times,
    to give its motion planner system enough data to work with.
    This results in smoother and faster axis motion. This is also
    called 'advanced streaming protocol based on counting
    characters' -- see Grbl Wiki.

    You can dynamically change the streaming method even
    during streaming, while running a job. The buffer fill
    percentage will reflect the change even during streaming.

    @param onoff
    Set to `True` to use incremental streaming. Set to `False` to
    use non-incremental streaming. The default on module startup
    is `False`.
    """
    self._incremental_streaming = value
    if self._incremental_streaming:
        self._wait_empty_buffer = True
    self.logger.debug('{}: Incremental streaming set to {}'.format(self.name, self._incremental_streaming))

def send_immediately(self, line):
    """
    G-Code command strings passed to this function will bypass
    buffer management and will be sent to Grbl immediately.
    Use this function with caution: Only send when you
    are sure Grbl's receive buffer can handle the data volume and

```

when it doesn't interfere with currently running streams.

Only send single commands at a time.

Applications of this method: manual jogging, coordinate settings etc.

@param line

A string of a single G-Code command to be sent. Doesn't have to be \n terminated.

"""

```
bytes_in_firmware_buffer = sum(self._rx_buffer_fill)
```

```
if bytes_in_firmware_buffer > 0:
```

```
    self.logger.error('Firmware buffer has {d} unprocessed bytes in it. Will not send {}'.format(bytes_in_firmware_buffer,
    return
```

```
if self.cmode == 'Alarm':
```

```
    self.logger.error('Grbl is in ALARM state. Will not send {}'.format(line))
    return
```

```
if self.cmode == 'Hold':
```

```
    self.logger.error('Grbl is in HOLD state. Will not send {}'.format(line))
    return
```

```
if "$#" in line:
```

```
    # The PRB response is sent for $# as well as when probing.
    # Regular querying of the hash state needs to be done like this,
    # otherwise the PRB response would be interpreted as a probe answer
    self.hash_state_requested = True
    return
```

```
self.preprocessor.set_line(line)
```

```
self.preprocessor.strip()
```

```
self.preprocessor.tidy()
```

```
self.preprocessor.parse_state()
```

```
self.preprocessor.override_feed()
```

```
self._iface_write(self.preprocessor.line + '\n')
```

```
def write(self, lines):
```

```
    """
```

G-Code command strings passed to this function will be appended to the current queue buffer, however a job is not started automatically. You have to call `job\_run()` to start streaming.

You can call this method repeatedly, e.g. for submitting chunks of G-Code, even while a job is running.

@param lines

A list or newline-separated string of G-Code commands.

"""

```
self._load_lines_into_buffer(lines)
```

```

def load_file(self, filename):
    """
    Pass a filename to this function to load its contents into the
    buffer.
    A job is not started automatically. Call `job_run` to start the job.

    @param filename
    A string giving the relative or absolute file path
    """
    with open(filename) as f:
        self._load_lines_into_buffer(f.read())

def job_run(self, linenr=None):
    """
    Run the current job, i.e. start streaming the current buffer
    from a specific line number.

    @param linenr
    If `linenr` is not specified, start streaming from the current
    buffer position (`self.current_line_number`).
    If `linenr` is specified, start streaming from this line.
    """
    if self.buffer_size == 0:
        self.logger.warning('{}: Cannot run job. Nothing in the buffer!'.format(self.name))
        return

    if linenr:
        self.current_line_number = linenr

    self.travel_dist_current = {}

    self._set_streaming_src_end_reached(False)
    self._set_streaming_complete(False)
    self._streaming_enabled = True
    self._current_line_sent = True
    self._set_job_finished(False)
    self._stream()

def job_halt(self):
    """
    Stop streaming. Grbl still will continue processing
    all G-Code in its internal serial receive buffer.
    """
    self._streaming_enabled = False

def job_new(self):
    """
    Start a new job. A "job" in our terminology means the buffer's
    contents. This function will empty the buffer, set the buffer
    position to 0, and reset internal state.

```

```
"""
```

```
del self.buffer[:]
self.buffer_size = 0
self._current_line_nr = 0
self._callback('on_line_number_change', 0)
self._callback('on_bufsize_change', 0)
self._set_streaming_complete(True)
self.job_finished = True
self._set_streaming_src_end_reached(True)
self._error = False
self._current_line = ""
self._current_line_sent = True
self.travel_dist_buffer = {}
self.travel_dist_current = {}
```

```
self._callback('on_vars_change', self.preprocessor.vars)
```

```
@property
```

```
def current_line_number(self):
    return self._current_line_nr
```

```
@current_line_number.setter
```

```
def current_line_number(self, linenr):
    if linenr < self.buffer_size:
        self._current_line_nr = linenr
        self._callback('on_line_number_change', self._current_line_nr)
```

```
def request_settings(self):
```

```
"""
```

```
This will send `$$` to Grbl and you will receive a callback with
the argument 1 "on_settings_downloaded", and argument 2 a dict
of the settings.
```

```
"""
```

```
self._iface_write('$$\n')
```

```
def do_buffer_stash(self):
```

```
"""
```

```
Stash the current buffer and position away and initialize a
new job. This is useful if you want to stop the current job,
stream changed $ settings to Grbl, and then resume the job
where you left off. See also `self.buffer_unstash()`.
"""
```

```
self.buffer_stash = list(self.buffer)
self.buffer_size_stash = self.buffer_size
self._current_line_nr_stash = self._current_line_nr
self.job_new()
```

```
def do_buffer_unstash(self):
```

```
"""
```

```
Restores the previous stashed buffer and position.
```

```
"""
```

```
self.buffer = list(self.buffer_stash)
self.buffer_size = self.buffer_size_stash
self.current_line_number = self._current_line_nr_stash
self._callback('on_bufsize_change', self.buffer_size)
```

```
def update_preprocessor_position(self):
    # keep preprocessor informed about current working pos
    self.preprocessor.position_m = list(self.cmpo)
    # self.preprocessor.target = list(self.cmpo)
```

```
def _preprocessor_callback(self, event, *data):
    if event == 'on_preprocessor_var_undefined':
        self.logger.critical(
            'HALTED JOB BECAUSE UNDEFINED VAR {}'.format(data[0])
        )
        self._set_streaming_src_end_reached(True)
        self.job_halt()
    else:
        self._callback(event, *data)
```

```
def _stream(self):
    if self._streaming_src_end_reached:
        return
```

```
if not self._streaming_enabled:
    return
```

```
if self.target == 'firmware':
    if self._incremental_streaming:
        self._set_next_line()
        if not self._streaming_src_end_reached:
            self._send_current_line()
        else:
            self._set_job_finished(True)
    else:
        self._fill_rx_buffer_until_full()
```

```
elif self.target == 'simulator':
    buf = []
    while not self._streaming_src_end_reached:
        self._set_next_line(True)
        if self._current_line_nr < self.buffer_size:
            buf.append(self._current_line)
```

```
# one line still to go
self._set_next_line(True)
buf.append(self._current_line)
```

```
self._set_job_finished(True)
self._callback('on_simulation_finished', buf)
```

```

def _fill_rx_buffer_until_full(self):
    while True:
        if self._current_line_sent:
            self._set_next_line()

        if not self._streaming_src_end_reached and self._rx_buf_can_receive_current_line():
            self._send_current_line()
        else:
            break

def _set_next_line(self, send_comments=False):
    progress_percent = int(100 * self._current_line_nr / self.buffer_size)
    self._callback('on_progress_percent', progress_percent)

    if self._current_line_nr < self.buffer_size:
        # still something in _buffer, pop it
        line = self.buffer[self._current_line_nr].strip()
        self.preprocessor.set_line(line)
        self.preprocessor.substitute_vars()
        self.preprocessor.parse_state()
        self.preprocessor.override_feed()
        self.preprocessor.scale_spindle()

        if send_comments:
            self._current_line = self.preprocessor.line + self.preprocessor.comment
        else:
            self._current_line = self.preprocessor.line

        self._current_line_sent = False
        self._current_line_nr += 1

        self.preprocessor.done()

    else:
        # the buffer is empty, nothing more to read
        self._set_streaming_src_end_reached(True)

def _send_current_line(self):
    if self._error:
        self.logger.error('Firmware reported error. Halting.')
        self._set_streaming_src_end_reached(True)
        self._set_streaming_complete(True)
        return

    self._set_streaming_complete(False)

    # +1 for \n which we will append below
    line_length = len(self._current_line) + 1
    self._rx_buffer_fill.append(line_length)
    self._rx_buffer_backlog.append(self._current_line)
    self._rx_buffer_backlog_line_number.append(self._current_line_nr)

```

```

self._iface_write(self._current_line + '\n')

self._current_line_sent = True
self._callback('on_line_sent', self._current_line_nr,
               self._current_line)

def _rx_buf_can_receive_current_line(self):
    rx_free_bytes = self._rx_buffer_size - sum(self._rx_buffer_fill)
    required_bytes = len(self._current_line) + 1 # +1 because \n
    return rx_free_bytes >= required_bytes

def _rx_buffer_fill_pop(self):
    if len(self._rx_buffer_fill) > 0:
        self._rx_buffer_fill.pop(0)
        processed_command = self._rx_buffer_backlog.pop(0)
        ln = self._rx_buffer_backlog_line_number.pop(0) - 1
        self._callback('on_processed_command', ln, processed_command)

    if self._streaming_src_end_reached and len(self._rx_buffer_fill) == 0:
        self._set_job_finished(True)
        self._set_streaming_complete(True)

def _iface_write(self, line):
    self._callback('on_write', line)
    if self._iface:
        num_written = self._iface.write(line)

def _onread(self):
    while self._iface_read_do:
        line = self._queue.get()

        if len(line) > 0:
            if line[0] == '<':
                self._update_state(line)

            elif line == 'ok':
                self._handle_ok()

            elif re.match(r'^\[G[0123] .*, line):
                self._update_gcode_parser_state(line)
                self._callback("on_read", line)

            elif line.startswith('[MSG:'):
                # nothing to do here
                pass

            elif re.match(r'^\[...:.*', line):
                self._callback('on_read', line)
                self._update_hash_state(line)

        if 'PRB' in line:

```

```

        # last line
        self._callback('on_hash_stateupdate', self.settings_hash)
        self.preprocessor.cs_offsets = self.settings_hash
        self._callback('on_probe', self.settings_hash['PRB'])

elif 'ALARM' in line:
    # grbl for some reason doesn't respond to ? polling
    # when there is an alarm due to soft limits
    self.cmode = 'Alarm'
    self._callback('on_stateupdate', self.cmode, self.cmpos, self.cwpos)
    self._callback('on_read', line)
    self._callback('on_alarm', line)

elif 'error' in line:
    # self.logger.debug("ERROR")
    self._error = True
    # self.logger.debug("%s: _rx_buffer_backlog at time of error: %s", self.name, self._rx_buffer_backlog)
    if len(self._rx_buffer_backlog) > 0:
        problem_command = self._rx_buffer_backlog[0]
        problem_line = self._rx_buffer_backlog_line_number[0]
    else:
        problem_command = 'unknown'
        problem_line = -1
    self._callback('on_error', line, problem_command, problem_line)
    self._set_streaming_complete(True)
    self._set_streaming_src_end_reached(True)

elif "Grbl " in line:
    self._callback('on_read', line)
    self._on_bootup()
    self.hash_state_requested = True
    self.request_settings()
    self.gcode_parser_state_requested = True

else:
    m = re.match(r'\$(.*)=(.*) \((.*)\)', line)
    if m:
        key = int(m.group(1))
        val = m.group(2)
        comment = m.group(3)
        self.settings[key] = {
            'val': val,
            'cmt': comment
        }
        self._callback('on_read', line)
        if key == self._last_setting_number:
            self._callback('on_settings_downloaded', self.settings)
    else:
        self._callback('on_read', line)
        # self.logger.info("{}: Could not parse settings: {}".format(self.name, line))

```

```

def _handle_ok(self):
    if not self.streaming_complete:
        self._rx_buffer_fill_pop()
        if not (self._wait_empty_buffer and len(self._rx_buffer_fill) > 0):
            self._wait_empty_buffer = False
            self._stream()

    self._rx_buffer_fill_percent = int(100 - 100 * (self._rx_buffer_size - sum(self._rx_buffer_fill)) / self._rx_buffer_size)
    self._callback('on_rx_buffer_percent', self._rx_buffer_fill_percent)

def _on_bootup(self):
    self._onboot_init()
    self.connected = True
    self.logger.debug('{}: Grbl has booted!'.format(self.name))
    self._callback('on_boot')

def _update_hash_state(self, line):
    line = line.replace(']', ' ').replace('[', ' ')
    parts = line.split(':')
    key = parts[0]
    tpl_str = parts[1].split(',')
    tpl = tuple([float(x) for x in tpl_str])
    self.settings_hash[key] = tpl

def _update_gcode_parser_state(self, line):
    m = re.match(r'\[G(\d) G(\d\d) G(\d\d) G(\d\d) G(\d\d) G(\d\d) M(\d) M(\d) M(\d) T(\d) F([\d.~]*?) S([\d.~]*?)\]', line)
    if m:
        self.gps[0] = m.group(1) # motionmode
        self.gps[1] = m.group(2) # current coordinate system
        self.gps[2] = m.group(3) # plane
        self.gps[3] = m.group(4) # units
        self.gps[4] = m.group(5) # dist
        self.gps[5] = m.group(6) # feed rate mode
        self.gps[6] = m.group(7) # program mode
        self.gps[7] = m.group(8) # spindle state
        self.gps[8] = m.group(9) # coolant state
        self.gps[9] = m.group(10) # tool number
        self.gps[10] = m.group(11) # current feed
        self.gps[11] = m.group(12) # current rpm
        self._callback('on_gcode_parser_stateupdate', self.gps)

        self.update_preprocessor_position()
    else:
        self.logger.error('{}: Could not parse gcode parser report: "{}".format(self.name, line))

def _update_state(self, line):
    m = re.match(r'<(.~*),MPos:(.~*),WPos:(.~*)>', line)
    if m is not None:
        # GRBL v0.9
        # <Idle,MPos:0.000,3.000,0.000,WPos:0.000,3.000,0.000>
        self.cmode = m.group(1)

```

```

mpos_parts = m.group(2).split(',')
wpos_parts = m.group(3).split(',')
self.cmpo = (float(mpos_parts[0]), float(mpos_parts[1]), float(mpos_parts[2]))
self.cwpos = (float(wpos_parts[0]), float(wpos_parts[1]), float(wpos_parts[2]))
else:
    # GRBL v1.1
    # <Idle|MPos:0.0000,0.0000,0.0000|Bf:15,128|FS:0.0,0|WCO:0.0000,0.0000,0.0000>
    m = re.match(r'<(.*?)\\MPos:(.*?)\\|', line)
    if m is not None:
        # machine position reported (when $10=1)
        self.cmode = m.group(1)
        mpos_parts = m.group(2).split(',')
        self.cmpo = (float(mpos_parts[0]), float(mpos_parts[1]), float(mpos_parts[2]))
    else:
        m = re.match(r'<(.*?)\\WPos:(.*?)\\|', line)
        if m is not None:
            # work position reported (when $10=0)
            self.cmode = m.group(1)
            wpos_parts = m.group(2).split(',')
            self.cwpos = (float(wpos_parts[0]), float(wpos_parts[1]), float(wpos_parts[2]))
        else:
            self.logger.error('{}: Could not parse MPos or WPos: "{}".format(self.name, line))
            return
# if we made it here, we parsed MPos or WPos or both

if (
    self.cmode != self._last_cmode or
    self.cmpo != self._last_cmpo or
    self.cwpos != self._last_cwpos
):
    self._callback(
        'on_stateupdate',
        self.cmode,
        self.cmpo,
        self.cwpos)
    if self.streaming_complete and self.cmode == 'Idle':
        self.update_preprocessor_position()
        self.gcode_parser_state_requested = True

if (self.cmpo != self._last_cmpo):
    if self.is_standstill:
        self._standstill_watchdog_increment = 0
        self.is_standstill = False
        self._callback('on_movement')
else:
    # no change in positions
    self._standstill_watchdog_increment += 1

if not self.is_standstill and self._standstill_watchdog_increment > 10:
    # machine is not moving
    self.is_standstill = True

```

```

        self._callback('on_standstill')

    self._last_cmode = self.cmode
    self._last_cmpos = self.cmpos
    self._last_cwpos = self.cwpos

def _load_line_into_buffer(self, line):
    self.preprocessor.set_line(line)
    split_lines = self.preprocessor.split_lines()

    for l1 in split_lines:
        self.preprocessor.set_line(l1)
        self.preprocessor.strip()
        self.preprocessor.tidy()
        self.preprocessor.parse_state()
        self.preprocessor.find_vars()
        fractionized_lines = self.preprocessor.fractionize()

        for l2 in fractionized_lines:
            self.buffer.append(l2)
            self.buffer_size += 1

    self.preprocessor.done()

def _load_lines_into_buffer(self, lines):
    if type(lines) is str:
        lines = lines.split('\n')

    for line in lines:
        self._load_line_into_buffer(line)

    self._callback('on_bufsize_change', self.buffer_size)
    self._callback('on_vars_change', self.preprocessor.vars)

def is_connected(self):
    if not self.connected:
        # self.logger.info("{}: Not yet connected".format(self.name))
        pass
    return self.connected

def _onboot_init(self):
    # called after boot. Mimics Grbl's initial state after boot.
    del self._rx_buffer_fill[:]
    del self._rx_buffer_backlog[:]
    del self._rx_buffer_backlog_line_number[:]
    self._set_streaming_complete(True)
    self._set_job_finished(True)
    self._set_streaming_src_end_reached(True)
    self._error = False
    self._current_line = ""
    self._current_line_sent = True

```

```

self._clear_queue()
self.is_standstill = False
self.preprocessor.reset()
self._callback('on_progress_percent', 0)
self._callback('on_rx_buffer_percent', 0)

def _clear_queue(self):
    try:
        junk = self._queue.get_nowait()
        self.logger.debug('Discarding junk %s', junk)
    except:
        # self.logger.debug("Queue was empty")
        pass

def _poll_state(self):
    while self._poll_keep_alive:
        self._counter += 1

        if self.hash_state_requested:
            self.get_hash_state()
            self.hash_state_requested = False

        elif self.gcode_parser_state_requested:
            self.get_gcode_parser_state()
            self.gcode_parser_state_requested = False

        else:
            self._get_state()

        time.sleep(self.poll_interval)

    self.logger.debug('{}: Polling has been stopped'.format(self.name))

def _get_state(self):
    self._iface.write('?')

def get_gcode_parser_state(self):
    self._iface_write('$G\n')

def get_hash_state(self):
    if self.cmode == 'Hold':
        self.logger.info('{}: $# command not supported in Hold mode.'.format(self.name))
        return

    self._iface_write('$#\n')

def _set_streaming_src_end_reached(self, a):
    self._streaming_src_end_reached = a

def _set_streaming_complete(self, a):
    self.streaming_complete = a

```

```
def _set_job_finished(self, a):
    self.job_finished = a
    if a:
        self._callback('on_job_completed')

def _default_callback(self, status, *args):
    print('DEFAULT CALLBACK', status, args)
```